behavior26: Fine-tuning $\pi_{0.5}$ with an Auxiliary Goal-Progress Head for the 2026 BEHAVIOR-1K Challenge

Technical report

Imaan Soltanalipour Jeffrey Le

November 2026

Abstract

We fine-tune the organisers' released $\pi_{0.5}$ `turning_on_radio` checkpoint with an auxiliary head that predicts how much of the task's BDDL goal is already satisfied, and test whether this auxiliary signal changes task success. Our submitted policy succeeds on **9 of 20** public test instances of `turning_on_radio` (45%). A pre-registered paired A/B against an identical fine-tune without the head finds **no measurable difference** (40/108 vs. 38/108 on training instances, $\Delta Q = +0.019$, 95% CI $[-0.091, +0.128]$, $p = 0.73$); the control also scores 9/20 on the public test set. We attempted one of the challenge's 100 tasks, so our overall challenge score is about 0.0045. All code, configs, logs, labels and weights are public; the whole project ran on rented GPUs for about $140.

Materials. Code, configs, training and evaluation logs: https://github.com/wimaan3/behavior26 (MIT). Weights: https://huggingface.co/imaan3/behavior26-pi05-radio (Gemma Terms of Use). Docker image: `wimaan3/behavior26-policy:armB-5999` (digest `sha256:84f7bd76...`). Section 5 gives the commands that reproduce our numbers.

1 Approach

Base policy. $\pi_{0.5}$ in openpi (`wensi-ai/openpi`, commit `0cc8e355`), a PaliGemma vision-language backbone with a flow-matching action expert that outputs 23-dimensional R1Pro actions. We start from the organisers' released `turning_on_radio` checkpoint (`pi05_b1k`). A full fine-tune needs more than 70 GB of GPU memory, so we freeze the vision-language backbone (SigLIP and the Gemma LLM) and train only the 0.43B-parameter action expert, without EMA (config `pi05_b1k_frozen_vlm`, 13.5 GB of training state).

Progress head. Our addition is a linear layer on the mean-pooled action-expert output tokens (`suffix_out`) that predicts a goal-progress value $p_t \in [0, 1]$, trained with binary cross-entropy. The total loss is $\mathcal{L} = \mathcal{L}_{\text{action}} + \lambda \mathcal{L}_{\text{progress}}$. The head reads the action-expert tokens rather than the vision-language prefix because only the suffix is recomputed at every denoising step at inference; the intent is that the shared representation must encode task progress, not just the next action. The head's output is not used at inference. We chose $\lambda = 0.15$ from measured gradient norms so the progress term contributes about 15–20% of the update (`analysis/lambda_calibration.py`).

Progress labels. Labels come from the dataset's per-step reward traces: $\text{satisfied}_t = s_0 + \sum_{\tau \le t} r_\tau$ goal units, divided by the task's goal-unit count D (`analysis/reward/labels.py`). The label is anchored at the start of each episode rather than assuming every demonstration finishes. `turning_on_radio` has $D = 1$, so its label is binary: 0 until the radio turns on, 1 afterwards (27.9% of frames). Labels for all 200 radio demonstrations (429,928 frames) are in `labels/` in the repository.

2 Training

Both arms were trained by one runner (`scripts/shot_one.sh` with `INIT_FROM=released STATS_FROM=released TASKS_CSV=turning_on_radio`) in one trainer call:

	Arm A (control)	Arm B (submitted)
Config	`pi05_b1k_frozen_vlm`	`pi05_b1k_frozen_vlm_progress`, $\lambda = 0.15$
Initial weights	released radio checkpoint	same; progress head initialised fresh
Data	200 radio demos, 429,928 frames	same, plus progress labels
Steps / batch / seed	6,000 / 32 / 42	same
Norm stats, prompt	released checkpoint's; task name	same
Action loss: start / 1k / 6k	0.080 / 0.031 / 0.015	0.080 / 0.032 / 0.022

Hardware: one NVIDIA RTX PRO 4500 (32 GB) on RunPod, 3.71 s per step, about $10.50 for both arms. Arm B's progress loss fell from 0.578 to 0.023 by step 1,000 and to about 0.001 after.

3 Evaluation and results

All evaluation used BEHAVIOR-1K v3.9.2 with the stock evaluator (`omnigibson.eval.eval`), the stock `RGBDFullResWrapper` (RGB, depth and proprioception only), and unmodified simulator settings.

A/B (pre-registered). The design, the analysis and the single allowed early-stop rule were fixed before any run (`docs/sessionE-2026-10-08-shot2/PLAN.md`): 27 training-split instances (10–36), 4 rollouts each, per arm, paired by instance, with no extension for significance.

Public test. One rollout per public test instance (301–320) for the entry, which was fixed as arm B before either run. Arm A was evaluated once afterwards on the same instances for internal comparison and was not submitted.

	Arm A (control)	Arm B (progress head)
Training instances 10–36, 4×27 rollouts	38/108 (35.2%)	40/108 (37.0%)
Public test 301–320, 1×20 rollouts	9/20 (45%)	**9/20 (45%)**

The paired difference on training instances is $\Delta Q = +0.019$, 95% CI $[-0.091, +0.128]$, $p = 0.73$ (t_{26}). The design could detect $|\Delta Q| \geq 0.11$. Outcomes are driven by the instance: 5 instances succeed almost every time for both arms and 7 never succeed for either. On the public test set, arm B succeeds on instances 304, 306–310, 316, 317 and 320; every failure ran to the 3,225-step limit with $Q = 0$. Rollout videos are linked from the repository README.

For reference only (not controlled: different instances, GPU and prompt), the released checkpoint scored 3/27 on instances 0–26 in our stack on 13 Sep. Both of our fine-tunes therefore complete the task far more often, but the difference comes from the radio fine-tune itself, not from the progress head.

4 Discussion

Why the head did not matter. On a $D = 1$ task the progress label only says "is the task done yet". The head learned this almost immediately, and with its loss near zero it sends almost no gradient into the shared trunk for the remaining 5,000 steps, so the two arms train nearly identically. The task on which our budget could train a policy that succeeds is the task where a progress head has the least to say. A graded task with $D > 1$ would test the idea properly. Our first attempt at that (*shot one*: coffee station and shoes, 10,000 steps from `pi05_base`) produced no task successes for either arm; its shoes A/B ($\Delta Q = +0.008$, $p = 0.68$) was measured at the floor and is uninformative.

Limitations. One task of 100; one public-test rollout per instance; 20 public-test rollouts cannot resolve small differences. Some A/B videos were lost to a disk-budget bug (164 of 216 survive); all per-rollout metrics were kept. The first public-test launch crashed on a host with NVIDIA driver 595.91 before finishing any rollout and was rerun unchanged on driver 580.159; both logs are in the repository.

5 Reproducing our results

1. Serve the policy (one GPU with bfloat16 support, about 6 GB of GPU memory). Either run the image:

```
docker run --gpus all -p 8000:8000 wimaan3/behavior26-policy:armB-5999
```

or build the server from source with the published weights:

```
git clone https://github.com/wimaan3/behavior26 && cd behavior26
git clone https://github.com/wensi-ai/openpi.git ../openpi
git -C ../openpi checkout 0cc8e355f7bac0976db1cc3139b1ff0379feea60
OPENPI_ROOT=../openpi scripts/apply_openpi_patches.sh
hf download imaan3/behavior26-pi05-radio --local-dir ../ckpt
cd ../openpi && uv sync && uv run scripts/b1k/serve_b1k.py \
  --robot b1k/R1Pro --task b1k/turning_on_radio --repo-id turning_on_radio \
  --port 8000 --policy.config pi05_b1k_frozen_vlm_progress --policy.dir ../ckpt
```

2. Evaluate (BEHAVIOR-1K v3.9.2):

```
python -m omnigibson.eval.eval --task-name turning_on_radio \
  --host 127.0.0.1 --port 8000 --mode public_test \
  --instance-indices 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 \
  --num-rollouts 1 --env-wrapper omnigibson.eval.wrappers.RGBDFullResWrapper \
  --robot-config r1pro.yaml --output-dir outputs/eval --write-video
```

The robot config we used is `configs/robot/r1pro.yaml`; the full submitted package, including the wrapper, is in `docs/submission-final-2026-10-16/`.

3. Retrain and rerun the A/B. `scripts/shot_one.sh` with the variables in Section 2 trains both arms from the released checkpoint; `scripts/eval_arms.sh` with `configs/experiments/002-radio-ab.yaml` evaluates them, and `analysis/compare.py` computes the paired result. Expected cost on RunPod: about $10 to train and $20 to evaluate.

Simulation is not bit-for-bit deterministic across GPUs and drivers, so reruns should match our success rates within sampling noise rather than rollout by rollout.